

Information and Coding (2026/27)

Lab work nº 1 — Due: 25 Oct 2026 — Discussion: 26 Oct 2026

Intro

- You may choose the programming language you find more appropriate to implement the project. You may even use different programming languages in different parts of the project. However, bear in mind that some of the tasks require computationally efficient languages, because running time and memory usage will be a factor of assessment.
- Use a github repository to manage the developement of your software.
- Your software must be provided in a way easy to install and be prepared to run at least in a Linux environment. Appropriate installation mechanisms (for example, makefiles), installation instructions and execution examples (for example, described in a readme file) must be provided.
- You will have some datasets available in eLearning to begin testing and tuning your codecs. Feel free to get more data and to use it. However, be careful not to over-specialize your programs in certain datasets, because we will also run performance tests using a private dataset.

Part I

1. Create a program, named `wav_hist`, able to compute and plot histograms of audio files in WAV format. This program should be prepared to
 - Plot the histogram of individual audio channels (for example, if the audio is stereo, the left channel L and/or the right channel R).
 - Plot the histogram of the average of the channels (i.e., the mono version, $(L + R)/2$; this is also known as the MID channel, used in audio coding) and the average difference of the channels (i.e., $(L - R)/2$, which is related to the SIDE channel, $L - R$)¹, when the audio is stereo.²
 - Provide coarser bins, i.e., instead of having an histogram bin for each different sample value, have bins that gather together $2, 4, 8, \dots, 2^k$ values.

¹In both cases, use integer division.

²It is possible to recover both channels, exactly, using the information in MID and SIDE. Can you find out how?

2. Implement a program, named `wav_quant`, to reduce the number of bits used to represent each audio sample (i.e., to perform uniform scalar quantization). Note that the result should be also a WAV file.
3. Implement a program, named `wav_cmp`, that prints, for each channel and for the average of the channels:
 - The average mean squared error between a certain audio file and its original version (also known as the L^2 norm).
 - The maximum per sample absolute error (also known as the L^∞ norm).
 - The signal-to-noise ratio (SNR) of a certain audio file in relation to its original version.
4. Implement a program, named `wav_effects`, that produces some audio effects. These can be, for example,
 - A single echo
 - Multiple echos
 - Amplitude modulation
 - Time-varying delays
 - ...

Part II

5. Develop a lossless audio codec, with the best performance that you can obtain. Note that performance means not only compression ratio, but also compression/decompression speed and memory usage. The encoder should accept a WAV file and produce the compressed version of it. The decoder should accept a compressed version and produce a WAV file exactly equal to the original WAV file. The calling format should be


```
wav_lossless_enc [ optional flags ] input.wav compressed_file
wav_lossless_dec compressed_file output.wav
```

 where `input.wav` and `output.wav` must be exactly equal (lossless compression).
6. Your codec should be compared with state-of-the-art lossless codeds, not only those specialized for audio coding (e.g., `flac`), but also others that are general purpose (e.g., `zstd`, `lzma`, ...). This comparison should include the compression ratio, but also compression/decompression time and memory usage.

Part III

7. Develop a lossy audio codec aiming at the best possible rate-distortion performance. The codec must be based on the Discrete Cosine Transform (DCT). All other design decisions are left to the

group and must be experimentally justified. The calling format should be

```
wav_lossy_enc [ optional flags ] input.wav compressed_file  
wav_lossy_dec compressed_file output.wav
```

8. Your codec should be compared with state-of-the-art lossy codeds, such as those of the MPEG family. This comparison should include the compression ratio and the corresponding SNR, but also compression/decompression time and memory usage.

Part IV

9. **Report 1:** Provide a report (12 pages at most), where you describe all the relevant steps and decisions taken in all the items of the work (for example, why a certain programming language was used to implement a certain part of the work; why did you choose a certain method; ...). When appropriate, include measures of processing time, compression ratios and corresponding errors introduced by the compression process. For this, use several audio files. This report should not include code. Instead, it should explain the main blocks of the system, why they were chosen and why do you think they are the most appropriate. Also, it should include a concise manual explaining how the software can be ran.
10. **Report 2:** Provide a report (6 pages at most), indicating all AI tools used to help you during the development of the work, together with a critical assessment of their effectiveness. It is expected that in this report you address questions such as: What was the problem? What did the AI propose? How did you verify it? Was it correct? What did you change? What did you learn?